Université de Lorraine

M1 Informatique - 2025/2026

Introduction à la Cryptographie

Marine Minier
marine.minier@loria.fr

Devoir Maison

Implémentation du cryptosystème de Paillier

Membres du groupe

Cissé Papa El Hadji G.
Keita Mahamadou
Abogounrin Ayath

Table des matières

1 Question 1 - Langage et bibliothèques **2**
1.1 Choix du langage 2
1.2 Bibliothèque pour les grands entiers 2

2 Question 2 - Nombre aléatoire cryptographiquement sûr **2**

3 Question 3 - Test de primalité probabiliste **2**
3.1 Test de Miller-Rabin 2
3.2 Bibliothèque utilisée 2

4 Question 4 - Implémentation de `ExpMod()` **3**
4.1 Algorithme 3
4.2 Version récursive vs itérative 3
4.3 Comparaison avec `pow()` 3

5 Question 5 - Implémentation de `KeyGen()` **3**

6 Question 6 – Implémentation de `Enc()` et `Dec()` **4**
6.1 Chiffrement `Enc(m, N)` 4
6.2 Déchiffrement `Dec(c, N,` φ`(N))` 4
6.3 Tests 5

7 Question 7 - Propriété homomorphique **5**
7.1 Définition 5
7.2 Preuve mathématique 5
7.3 Implémentation 5
7.4 Tests et application 5

Question 1 - Langage et bibliothèques

Choix du langage

Nous avons choisi **Python 3**. Python gère nativement les entiers de taille arbitraire (type `int`) sans bibliothèque externe : les opérations d'addition, soustraction, multiplication, division entière, modulo, exponentiation (`**`) et exponentiation modulaire (`pow(a, b, n)`) sont toutes disponibles nativement et fonctionnent sur des entiers de milliers de bits sans débordement.

Bibliothèque pour les grands entiers

Contrairement à des langages comme C (qui nécessite la bibliothèque GMP), Python intègre nativement l'arithmétique en précision arbitraire. Les opérations supportées incluent notamment :

- Addition, soustraction, multiplication : `+`, `-`, `*`
- Division entière et modulo : `//`, `%`
- Exponentiation modulaire : `pow(a, b, n)`
- Inverse modulaire : `pow(a, -1, n)` (Python 3.8+)

Question 2 - Nombre aléatoire cryptographiquement sûr

Un nombre aléatoire **cryptographiquement sûr** est un nombre généré par un CSPRNG (*Cryptographically Secure Pseudo-Random Number Generator*). Il doit satisfaire deux propriétés :

- **Imprévisibilité** : un attaquant ne peut pas deviner les prochaines valeurs même en connaissant les précédentes.
- **Uniformité statistique** : toutes les valeurs sont équiprobables.

En Python, le module `secrets` (bibliothèque standard) fournit la fonction `secrets.randbits(k)` qui génère un entier aléatoire de k bits via l'entropie du système d'exploitation (`/dev/urandom` sous Linux).

Question 3 - Test de primalité probabiliste

Test de Miller-Rabin

Le test habituellement utilisé pour tester la primalité d'un grand nombre est le **test de Miller-Rabin**. Ce test est dit **probabiliste** car il ne peut pas prouver avec certitude absolue qu'un nombre est premier : il peut uniquement prouver avec certitude qu'un nombre est *composé*. Lorsqu'un nombre passe le test pour k bases aléatoires, la probabilité qu'il soit premier est très élevée, mais pas garantie à 100 % : la probabilité d'erreur (faux positif) est au plus 4^{-k}.

Bibliothèque utilisée

En Python, on utilise `sympy.isprime(n)` de la bibliothèque **sympy**, qui combine le test de Miller-Rabin avec d'autres tests pour une fiabilité très élevée. La bibliothèque **gmpy2** propose également `gmpy2.is_prime(n)`.

Question 4 - Implémentation de `ExpMod()`

Algorithme

La fonction `ExpMod(c, a, N)` implémente l'exponentiation binaire décrite en Annexe A. Elle suit les trois cas :

$$\text{puissance}(g, a) = \begin{cases} g & \text{si } a = 1 \\ \text{puissance}(g^2,\ a/2) & \text{si } a \text{ est pair} \\ g \times \text{puissance}(g^2,\ (a-1)/2) & \text{si } a > 2 \text{ est impair} \end{cases}$$

Pour l'appliquer au contexte de Paillier, tous les calculs intermédiaires sont effectués modulo N, comme indiqué dans le sujet.

Version récursive vs itérative

Nous avons d'abord implémenté une version récursive `exp_mod_recursive()`. Cependant, Python impose une limite de récursion par défaut à 1000 appels, ce qui est insuffisant pour des exposants de 1024 bits ou plus. Augmenter cette limite via `sys.setrecursionlimit()` ne règle pas le problème en profondeur : la pile mémoire du système d'exploitation reste limitée, et un dépassement peut provoquer un crash de type `SegmentationFault`, plus grave qu'une simple `RecursionError`. Il s'agit d'un problème de **robustesse** et non de sécurité cryptographique.

C'est un exemple classique où la récursion est utile pour comprendre l'algorithme, mais l'itération est plus adaptée à l'implémentation. Nous avons donc opté pour une version **itérative** qui parcourt les bits de l'exposant du poids faible vers le poids fort :

```
def ExpMod(c, a, N):
    if a == 0:
        return 1
    res = 1
    base = c % N
    while a > 0:
        if a % 2 == 1:    # bit courant = 1
            res = (res * base) % N
        base = (base * base) % N
        a = a // 2
    return res
```

Comparaison avec `pow()`

Python implémente cette méthode nativement via `pow(base, exp, mod)` à trois arguments. Nous avons comparé notre implémentation à `pow()` sur 10 grands nombres (exposants, bases et modulo de 256 bits) : les résultats sont identiques dans tous les cas (**voir test.txt**).

Question 5 - Implémentation de `KeyGen()`

La fonction `KeyGen()` génère deux nombres premiers indépendants p et q de 1024 bits chacun à l'aide du générateur cryptographiquement sûr `secrets.randbits(1024)`, en

forçant le bit de poids fort à 1 pour garantir exactement 1024 bits. La primalité est vérifiée avec `sympy.isprime()`. On s'assure que $p \neq q$. La fonction retourne :

— la clef publique $pk = N = p \times q$ (un entier d'environ 2048 bits)
— la clef privée $sk = \varphi(N) = (p-1)(q-1)$

```
def KeyGen(bits=1024):
    p = generer_grand_nombre_premier(bits)
    q = generer_grand_nombre_premier(bits)
    while q == p:
        q = generer_grand_nombre_premier(bits)
    N = p * q
    phi_N = (p - 1) * (q - 1)
    return N, phi_N
```

La clef publique N obtenue fait 2047 ou 2048 bits, ce qui respecte le seuil de sécurité minimal recommandé de 2048 bits mentionné dans le sujet.

Question 6 – Implémentation de `Enc()` et `Dec()`

Chiffrement `Enc(m, N)`

La fonction `Enc(m, N)` chiffre un message m à l'aide de la clé publique N. Elle génère un entier aléatoire r cryptographiquement sûr avec `secrets.randbits(2048)`, en s'assurant que $0 < r < N$. Elle calcule ensuite :

$$c = (1+N)^m \cdot r^N \mod N^2$$

en utilisant notre fonction `ExpMod()` itérative pour les deux exponentiations modulaires.

```
def Enc(m, N):
    r = secrets.randbits(2048)
    while r <= 0 or r >= N:
        r = secrets.randbits(2048)
    c = (ExpMod(1 + N, m, N**2) * ExpMod(r, N, N**2)) % (N**2)
    return c, r
```

Déchiffrement `Dec(c, N, φ(N))`

La fonction `Dec(c, N, phi_N)` déchiffre un message à partir du chiffré c et de la clé privée $\varphi(N)$. Elle retrouve d'abord r via :

$$r = c^{N^{-1} \bmod \varphi(N)} \bmod N$$

où l'inverse $N^{-1} \bmod \varphi(N)$ est calculé via notre fonction `inverse_mod()` basée sur l'algorithme d'Euclide étendu. Elle calcule ensuite :

$$m = \frac{c \cdot r^{-N} \bmod N^2 - 1}{N}$$

en utilisant également `ExpMod()` pour les exponentiations. La fonction accepte un paramètre optionnel `r` pour permettre la réutilisation dans le déchiffrement homomorphique (voir Question 7) :

```
def Dec(c, N, phi_N, r=None):
    N2 = N**2
    if r is None:
        exp_r = inverse_mod(N, phi_N)
        r = ExpMod(c, exp_r, N)
    r_inv = inverse_mod(r, N2)
    r_invN = ExpMod(r_inv, N, N2)
    m = ((c * r_invN) % N2 - 1) // N
    return m
```

Tests

Nous avons vérifié le bon fonctionnement sur **100 messages aléatoires** de taille arbitraire (entre 1 et $N-1$) : le message déchiffré est égal au message original dans tous les cas.

Voir **test.txt**

Question 7 - Propriété homomorphique

Définition

Le cryptosystème de Paillier est un **homomorphisme additif** : le produit de deux chiffrés correspond, une fois déchiffré, à la somme des deux messages clairs. Formellement, si $c_1 = \text{Enc}(m_1, r_1)$ et $c_2 = \text{Enc}(m_2, r_2)$, alors :

$$c_1 \times c_2 \bmod N^2 = \text{Enc}(m_1 + m_2,\ r_1 \cdot r_2)$$

Preuve mathématique

Cela découle directement de la formule de chiffrement :

$$\begin{aligned} c_1 \times c_2 &= (1+N)^{m_1} \cdot r_1^N \cdot (1+N)^{m_2} \cdot r_2^N \pmod{N^2} \\ &= (1+N)^{m_1+m_2} \cdot (r_1 \cdot r_2)^N \pmod{N^2} \end{aligned}$$

Ce qui est exactement la formule de $\text{Enc}(m_1 + m_2)$ sous l'aléa $r_1 \cdot r_2$.

Implémentation

La fonction `homomorphic_add()` réutilise directement `Dec()` grâce au paramètre optionnel `r`, sans dupliquer la logique de déchiffrement :

```
def homomorphic_add(c1, c2, r1, r2, N, phi_N):
    c_sum = (c1 * c2) % (N**2)
    r_combined = (r1 * r2) % N
    return Dec(c_sum, N, phi_N, r=r_combined)
```

Tests et application

Pour chaque test, nous avons :

1. Chiffré m_1 et m_2 séparément avec `Enc()` pour obtenir (c_1, r_1) et (c_2, r_2)

2. Calculé $c_{\text{sum}} = c_1 \times c_2 \bmod N^2$
3. Déchiffré c_{sum} en passant l'aléa combiné $r_1 \cdot r_2 \bmod N$ à `Dec()`
4. Vérifié que le résultat est égal à $m_1 + m_2 \bmod \varphi(N)$

Ce test a été répété **10 fois** avec des messages aléatoires distincts.

Voir **test.txt**

Cette propriété trouve des applications concrètes dans des domaines comme le **vote électronique** : chaque électeur envoie son vote chiffré, le serveur multiplie tous les chiffrés ensemble, et en déchiffrant une seule fois on obtient le total des votes, sans jamais voir les votes individuels.